



Disciplina ATP	Curso Sistemas de Informação
Professora Ivete Marjorie R. Machado	
Atividade Arquivos em C#	

Introdução

Os arquivos TXT's podem ser uma excelente forma de trafegar dados entre aplicações. Sabemos que agora estamos no mundo do XML, mas na condição de alocação do arquivo para uma futura importação, ou até na leitura / extração de informações de portas seriais, o TXT atende bem a demanda para uma futura importação a um banco de dados.

Na plataforma .NET os arquivos são representados por objetos. No entanto, ao contrário de linguagens como o C, em que as operações de leitura e escrita em arquivos são diretas, em C# a leitura e escrita de arquivo é abstraída em termos de *streams*. Uma *stream* representa um fluxo de informação. Esse fluxo de informação vem de algum lado, não importa de onde, ou está sendo direcionado para algum lado.

1) Streams

Um stream é uma representação abstrata de um dispositivo serial. Um dispositivo como uma impressora, um local da memória ou qualquer objeto que suporte a leitura e a escrita nele de modo linear.

2) Classes File e Directory

2.1 Métodos úteis da classe "File":

Método	Descrição
Create ()	Criar um arquivo em um determinado caminho
Open ()	FileStream no caminho especificado
Copy ()	Copiar arquivo em um local específico

2.2 Métodos úteis da classe "Directory"

Método	Descrição
Delete ()	Exclui o diretório e todos os arquivos nele contido
Move ()	Mover um diretório para um novo local.
GetFiles()	Retorna uma array de objetos File

3) Classe FileInfo

A classe FileInfo não possui métodos estáticos e pode ser usada apenas em objetos instanciados. O objeto FileInfo é como se fosse um arquivo em um disco.

OBS: Para escrevermos ou lermos um arquivo, é necessário criarmos um objeto *Stream*. Vamos criar um objeto FileInfo, indicando o nome do arquivo e seu diretório.

```
FileInfo aFile = new FileInfo (@"c:/teste.txt");
```



Diferente da classe `FileInfo`, que não é uma classe estática, a classe `File` requer um parâmetro de string especificando a localização do arquivo para cada chamada de método.

OBS: As duas chamadas abaixo efetuam o mesmo resultado.

```
FileInfo aFile = new FileInfo("teste.txt");  
If (aFile.Exists)  
    Console.WriteLine("Esse arquivo existe");  
  
If (File.Exists("teste.txt"))  
    Console.WriteLine("Esse arquivo existe");
```

É sensato que você utilize a classe estática `File` se você estiver realizando uma **única** chamada de método do objeto. Caso a chamada do seu aplicativo estiver realizando várias operações em um arquivo, seria mais sensato instanciar o objeto `FileInfo` e usar os seus métodos, economizando mais tempo porque o objeto já estará referenciando o arquivo correto no sistema de arquivos, enquanto uma classe estática terá de encontrá-lo a cada vez.

`FileInfo` e `File` são similares, assim como `DirectoryInfo` e `Directory`.

ATENÇÃO: A grande parte dos métodos `FileInfo` parecem com os métodos `File`.

4) Hierarquia de Streams

Classe	Descrição
Stream	Representa a classe-base de onde todas as <i>streams</i> básicas derivam.
FileStream	Representa uma <i>stream</i> associada a um arquivo, sendo utilizada por outras <i>streams</i> para realizar leituras e escritas para disco.
StreamReader	Implementa uma <i>stream</i> que permite ler dados sobre a forma de texto de uma <i>stream</i> básica. No entanto, também possui um construtor que permite associar diretamente a <i>stream</i> a um arquivo.
StreamWriter	Implementa uma <i>stream</i> que permite escrever dados sobre a forma de texto de uma <i>stream</i> básica. No entanto, também possui um construtor que permite associar diretamente a <i>stream</i> a um arquivo.
BinaryReader	Trata-se de uma <i>stream</i> que permite ler dados, sob a forma binária, de uma <i>stream</i> básica (uma instância de <code>Stream</code>)
BinaryWriter	Trata-se de uma <i>stream</i> que permite escrever dados, sob a forma binária, de uma <i>stream</i> básica (uma <code>Stream</code>)



Exemplo:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.IO;
using System.Text;

namespace readwriteapp
{
    class Program
    {
        static void Main(string[] args)
        {
            Int64 i;
            try
            {
                //Abrir o arquivo
                StreamWriter valor = new
                StreamWriter("arquivo1.txt", true, Encoding.ASCII);
                //Loop para implementação dos numeros no TXT.
                for (i = 0; i < 10; i++)
                {
                    valor.Write(i);
                }
                //Fecha o arquivo
                valor.Close();
            }
            catch (Exception e)
            {
                Console.WriteLine("Exception: " + e.Message);
            }
            finally
            {
                Console.WriteLine("Parabéns, agora mais um arquivo!");
            }
            Console.Write("\n\nPressione qualquer tecla para sair...");
            Console.ReadKey(true);
        }
    }
}
```

Exercício

1- Analise cada código abaixo e comente o que cada linha executa.

a)

```
using System;
using System.IO;

namespace csharp_arquivos
{
    class TextFileWriter
    {
        static void Main(string[] args)
        {
            TextWriter tw = new StreamWriter("dados.txt");
            tw.WriteLine(DateTime.Now);
            tw.Close();
        }
    }
}
```



```
}  
b)  
using System;  
using System.IO;  
  
namespace csharp_arquivo2  
{  
    class TextFileReader  
    {  
        static void Main(string[] args)  
        {  
            TextReader tr = new StreamReader("dados.txt");  
            Console.WriteLine(tr.ReadLine());  
            tr.Close();  
        }  
    }  
}
```

2- Faça um programa que gere um **ArrayList** de tamanho aleatório (no mínimo 50 elementos) preenchidos com números inteiros aleatórios. Ordene essa lista e armazene seus elementos no arquivo EXERCICIO.TXT.

3- Faça um programa que leia o arquivo EXERCICIO.TXT gerado no exercício anterior e gere um arquivo EXERCICIO2.TXT invertido. Detalhe: você não poderá usar o método REVERSE do **ArrayList**.

4- Crie uma classe Aluno com os seguintes dados (Nome, matrícula, telefone, CPF, RG e curso). Leia os dados de diversos alunos, armazenando-os em uma lista. Ao terminar de inserir, imprima todos os alunos armazenados na lista.

5- Crie uma classe Aluno com os seguintes dados (Nome, matrícula, telefone, CPF, RG e curso). Leia os dados de diversos alunos, armazenando-os em uma lista. Ao sair do programa, você deve armazenar os dados no arquivo **ALUNOS.TXT**. Ao entrar no programa, seu programa deve ler os dados do arquivo e armazená-los novamente na lista. Não há limite para a quantidade de alunos.

6- Crie uma classe Contato com os seguintes dados (Nome, telefone, celular, e-mail). Crie um programa semelhante a uma AGENDA que permita inserir, remover, alterar, pesquisar e listar (imprimir) contatos. Seu programa deve usar o arquivo **AGENDA.TXT** para armazenar os dados de seus contatos. Internamente, use um **ArrayList** para armazenar os dados dos contatos.